

基于导向滤波优化的暗 通道先验去雾算法的硬件实 现

目录

CONTENTS

1

去雾算法的背景及概要

2

基于C++的算法分析与验证

3

硬件实现架构及仿真

4

Reference

1

PART 01

去雾算法的背景及概要

去雾算法的背景及概要

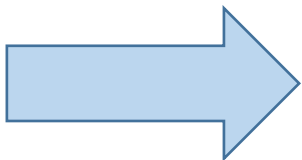
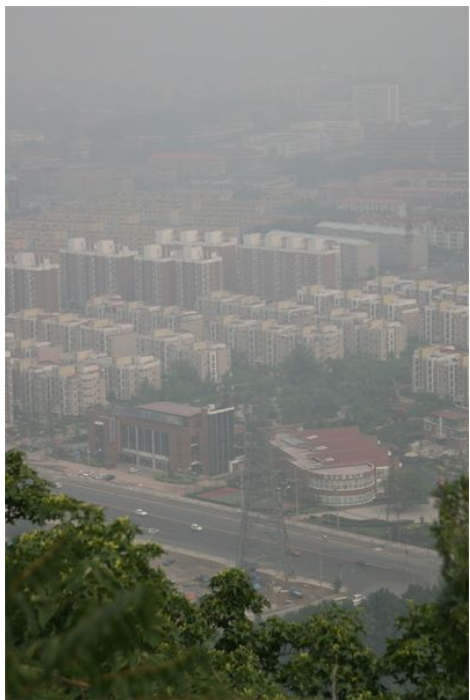
Haze Images



- Low visibility
- Faint colors

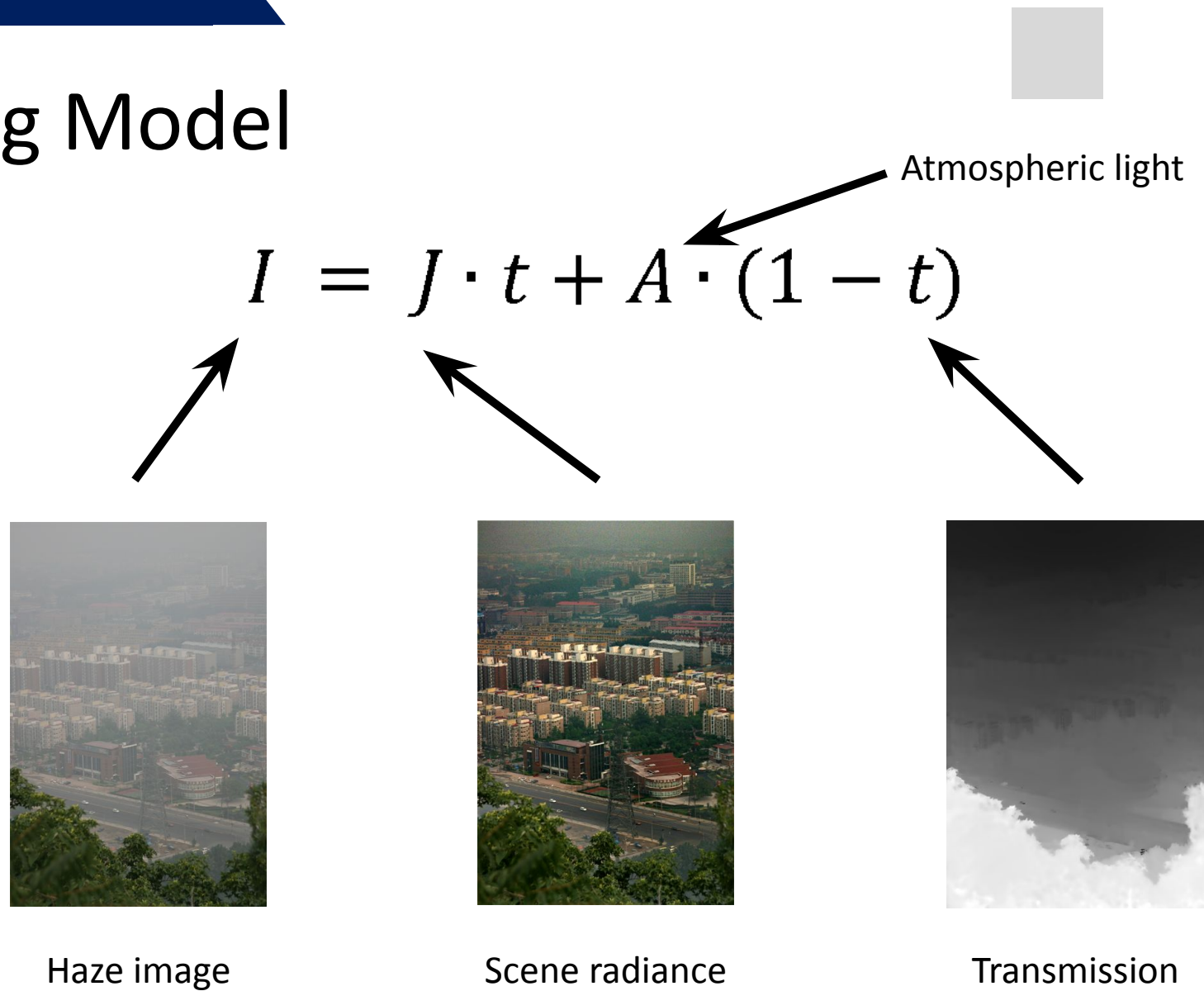
去雾算法的背景及概要

Goals of Haze Removal



- Scene restoration
- Depth estimation

Haze Imaging Model



Dark Channel

- min(rgb, local patch)
 - min(r,g,b)
 - min(local patch) = min filter

$$J^{\text{dark}}(\mathbf{x}) = \min_{y \in \Omega(\mathbf{x})} \left(\min_{c \in \{r,g,b\}} J^c(y) \right)$$

- J^c : color channel of J
- J^{dark} : dark channel of J



去雾算法的背景及概要

A Superising Observation

Haze-free



去雾算法的背景及概要

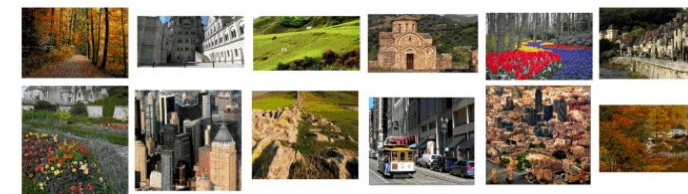
A Supervising Observation

Haze-free



去雾算法的背景及概要

A Surprising Observation

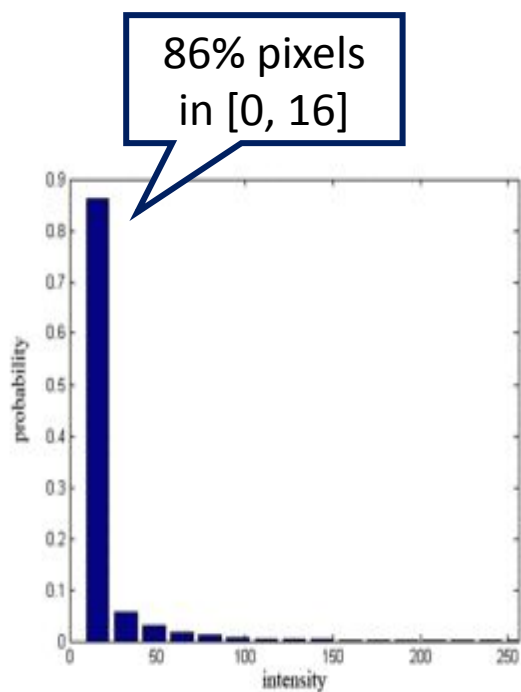


(a)

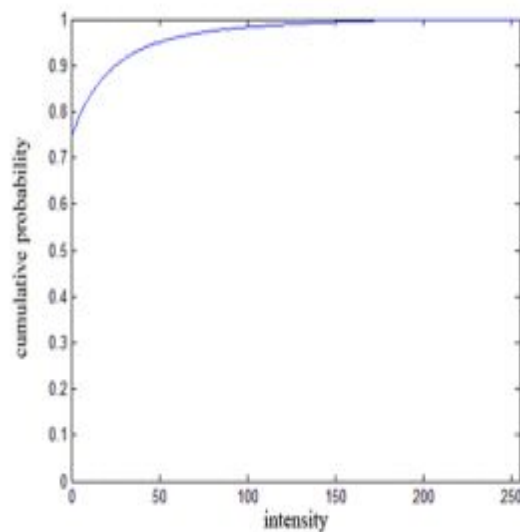


(b)

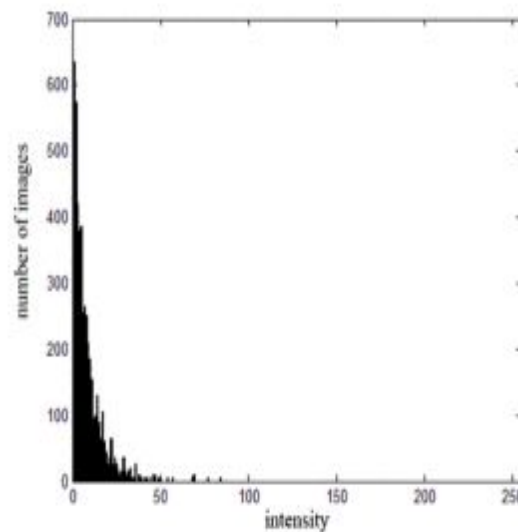
5,000 haze-free images



(a)



(b)



(c)

Dark Channel Prior

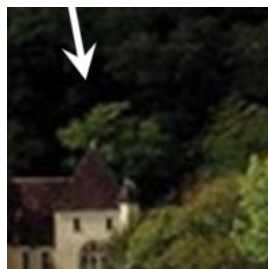
- For outdoor haze-free images

$$\min_{c \in \{r,g,b\}} J^c(y) \rightarrow 0$$

去雾算法的背景及概要

What makes it dark?

- Shadow



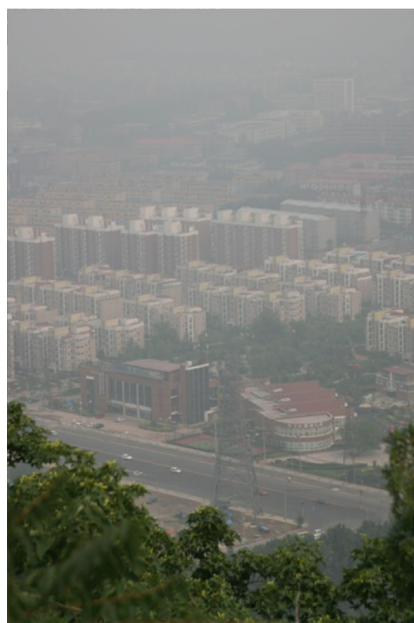
- Colorful object



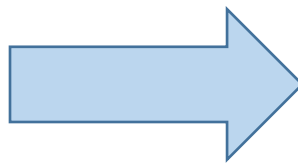
- Black object



Dark Channel of Hazy Image



haze image



dark channel

- The dark channel is no longer dark.

Dark Channel of Hazy Image

Haze imageing model

$$I = J \cdot t + A \cdot (1 - t)$$

Normalize

$$\frac{I^c}{A^c} = \frac{J^c}{A^c} t + 1 - t$$

Compute dark channel

$$\min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{I^c}{A^c} \right) = \left\{ \min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{J^c}{A^c} \right) \right\} t + 1 - t$$

Transmission Estimation

Dark Channel Prior

$$\min_{c \in \{r, g, b\}} J^c(y) \rightarrow 0$$

Compute dark channel

$$\min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{I^c}{A^c} \right) = \left\{ \min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{J^c}{A^c} \right) \right\}^t + 1 - t \rightarrow 0$$

Transmission Estimation

Estimate transmission

$$t = 1 - \min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{I^c}{A^c} \right)$$

Compute dark channel



$$\min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{I^c}{A^c} \right) = \left\{ \min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{J^c}{A^c} \right) \right\} t + 1 - t$$

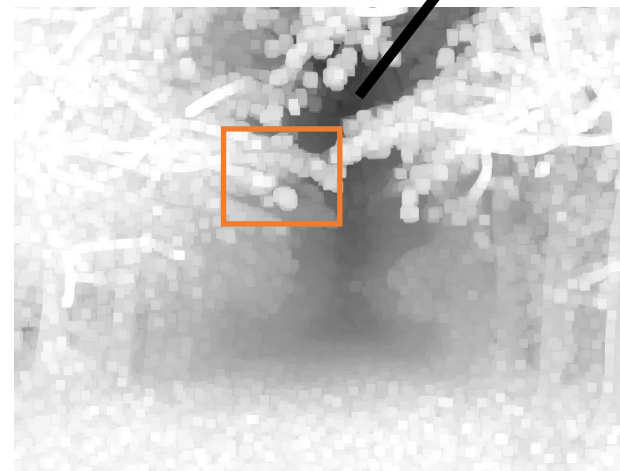
Transmission Estimation

Estimate transmission

$$t = 1 - \min_{y \in \Omega(x)} \left(\min_{c \in \{r, g, b\}} \frac{I^c}{A^c} \right)$$



input I



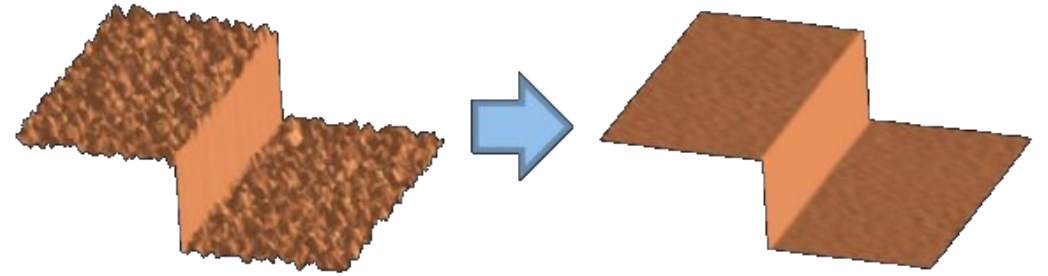
estimated t



Transmission Optimization

Introduction

- Edge-preserving filtering
 - An important topic in computer vision
 - Denoising, image smoothing/sharpening, texture decomposition, HDR compression, image abstraction, optical flow estimation, image superresolution, feature smoothing...



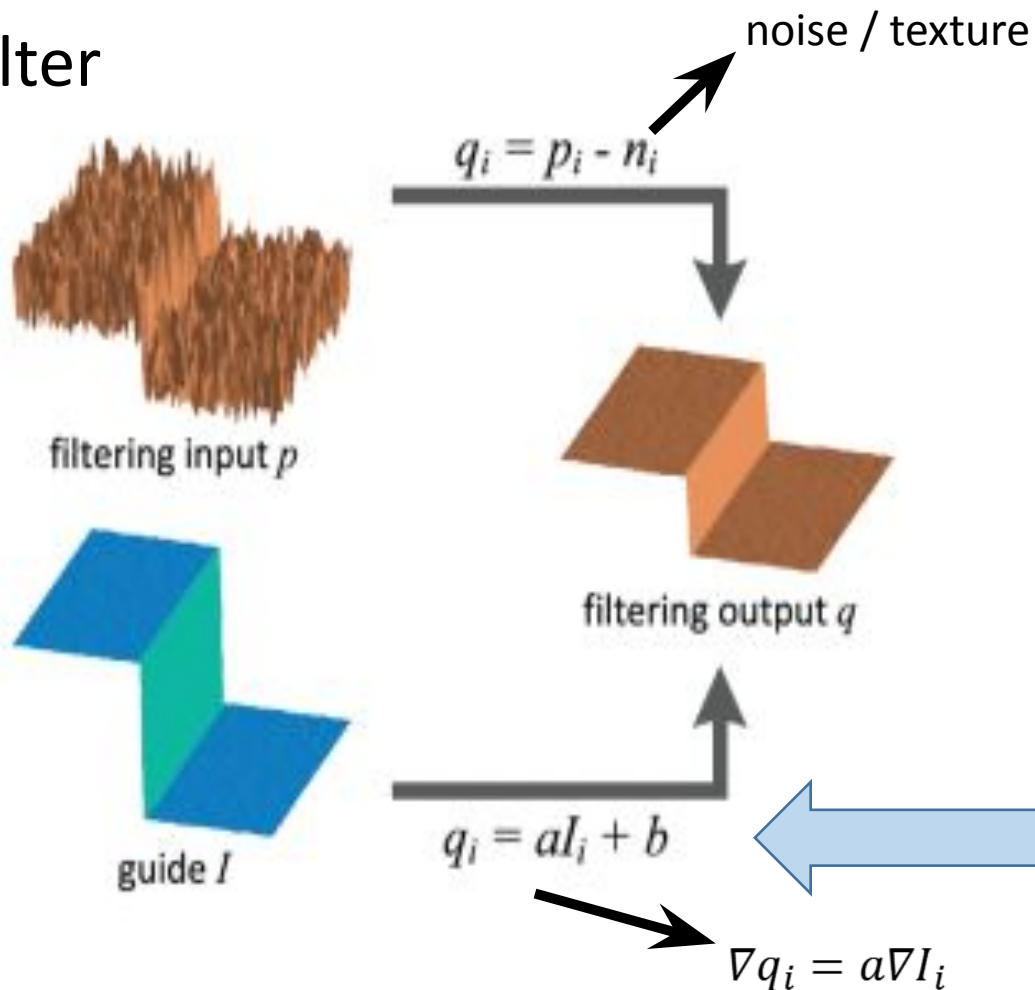
Transmission Optimization

Introduction

- Our target - to design a filter
 - Edge-preserving filtering
 - Non-iterative
 - $O(1)$ time, fast and non-approximate
 - No gradient distortion

Transmission Optimization

Guided filter

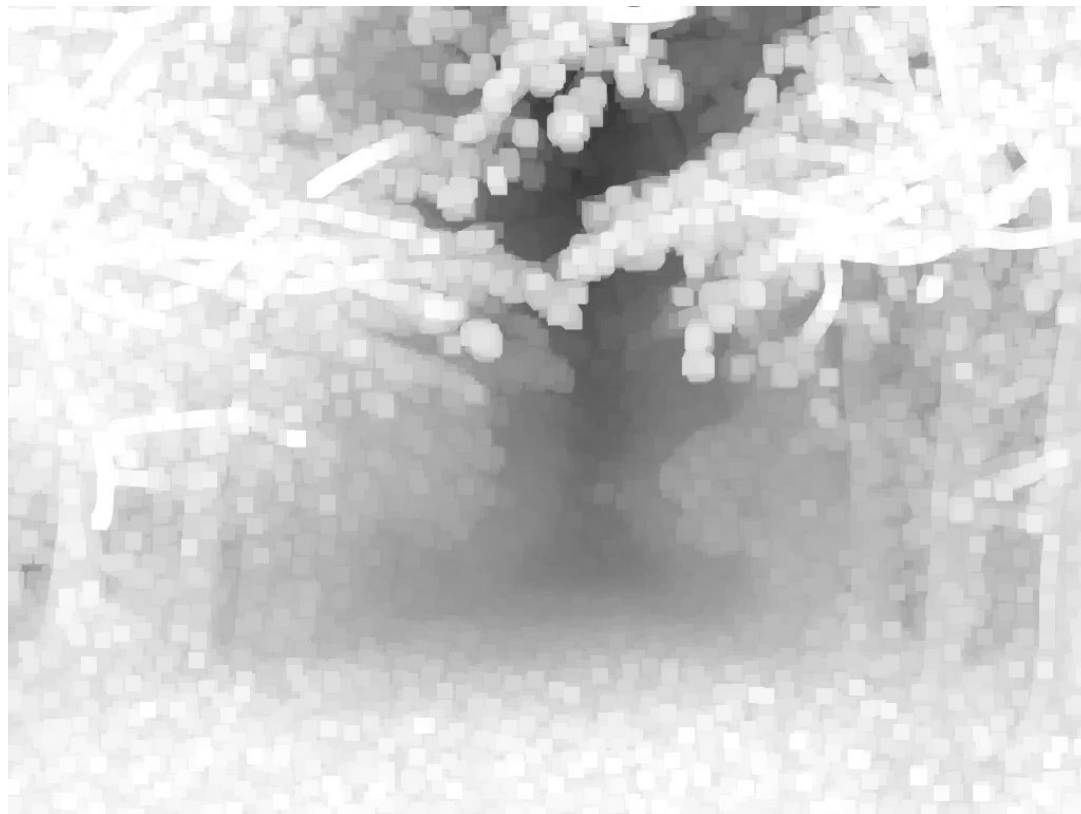


$$\min_{(a,b)} \sum_i (aI_i + b - p_i)^2 + \varepsilon a^2$$

Linear regression

$$\begin{cases} a = \frac{\text{cov}(I, p)}{\text{var}(I) + \varepsilon} \\ b = \bar{p} - a\bar{I} \end{cases}$$

Transmission Optimization



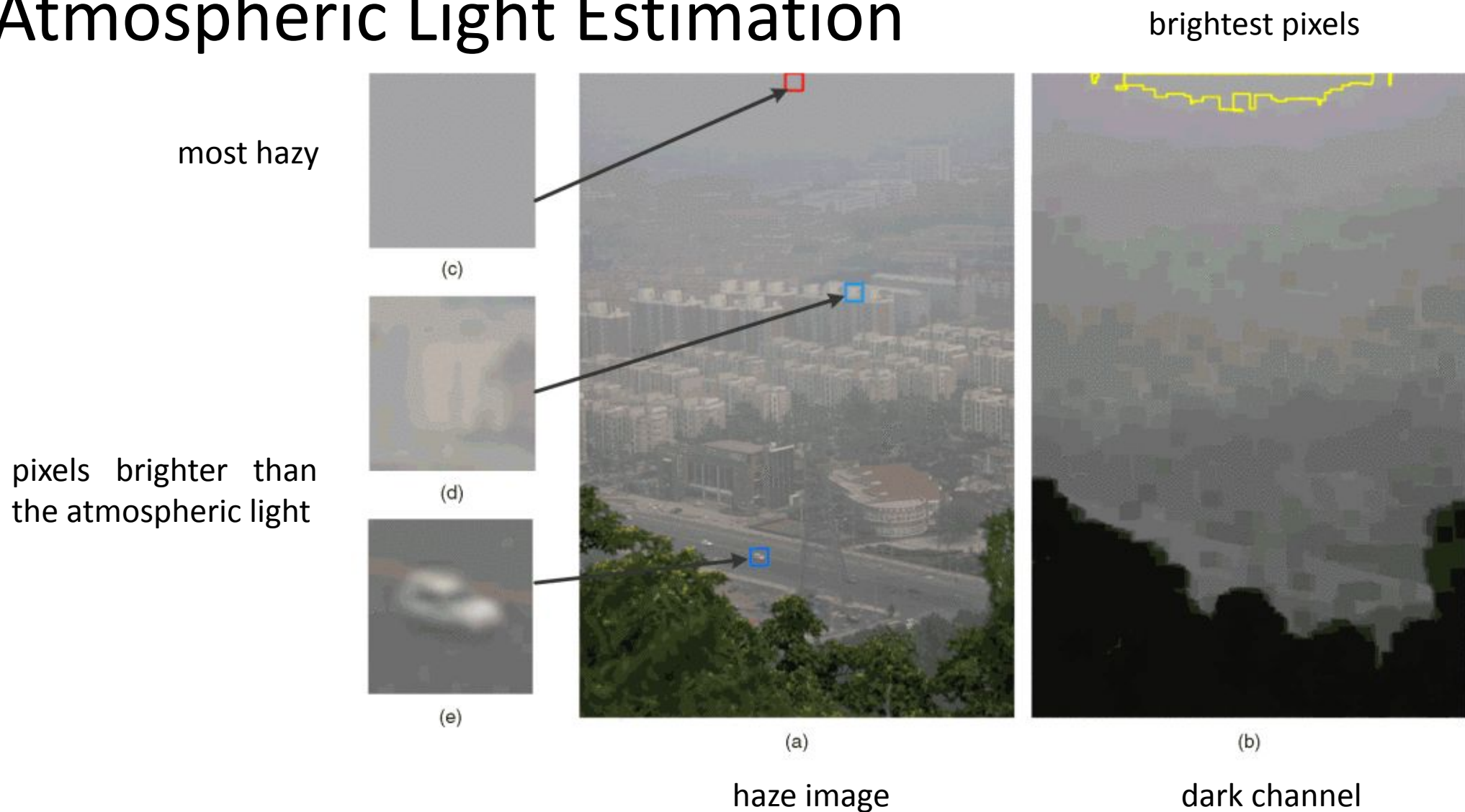
before optimization

Transmission Optimization

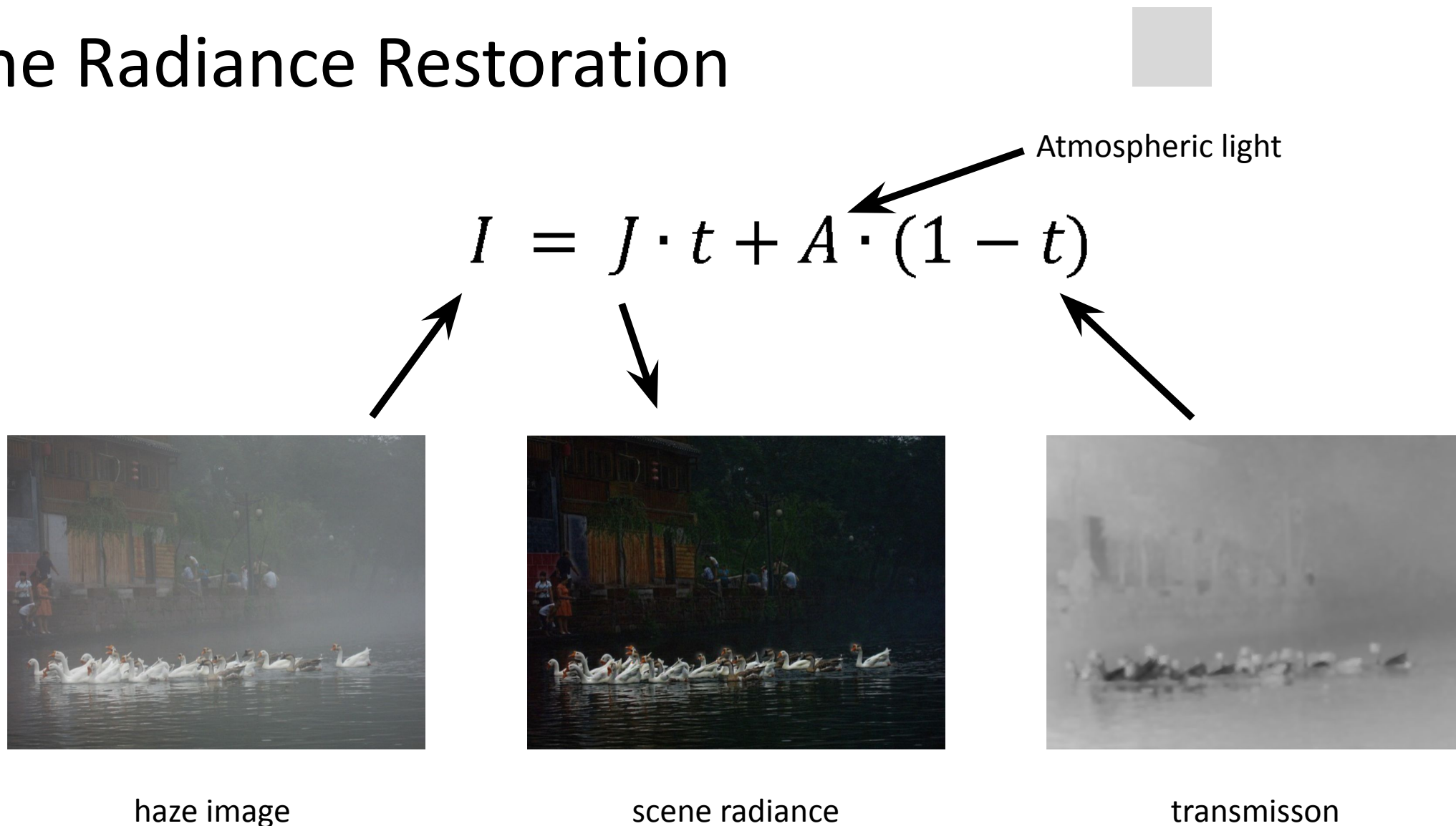


after optimization

Atmospheric Light Estimation

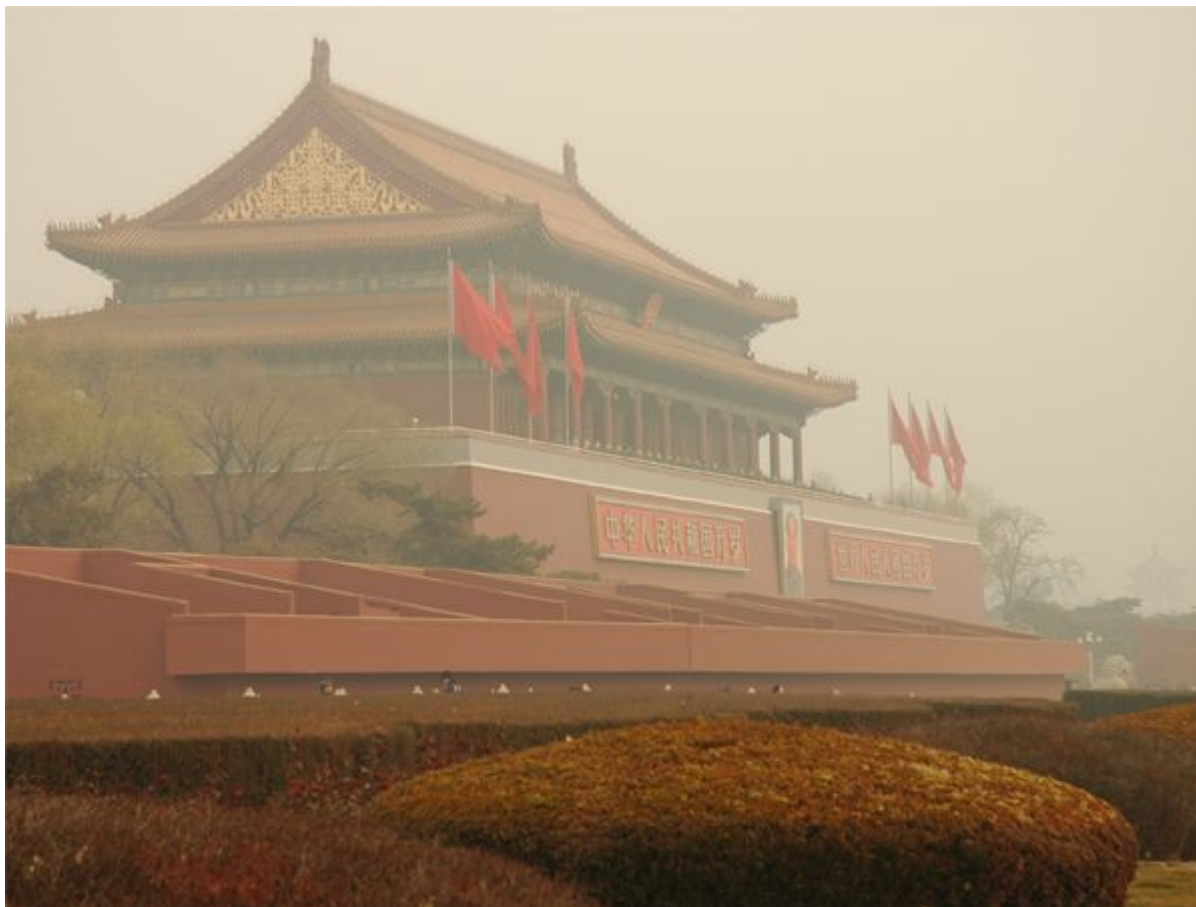


Scene Radiance Restoration



去雾算法的背景及概要

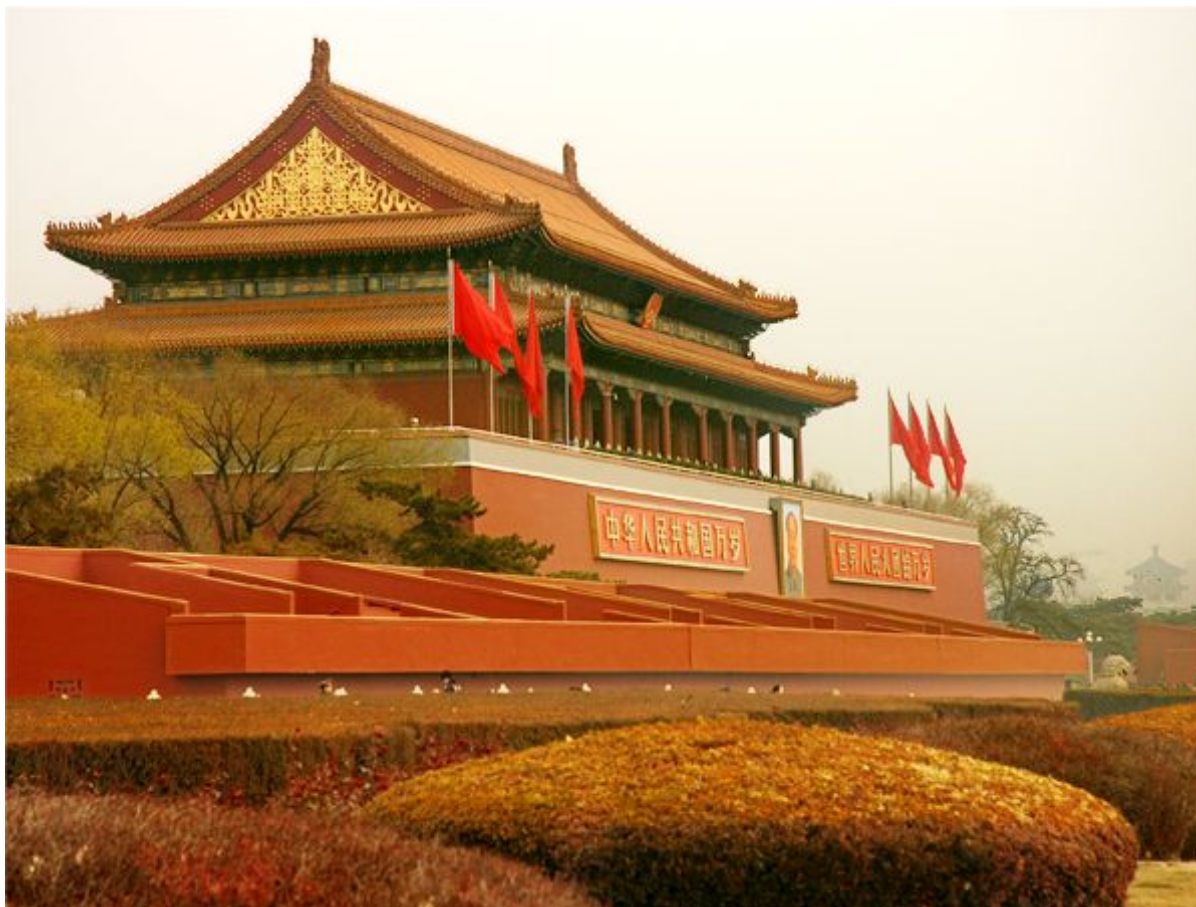
Result



input

去雾算法的背景及概要

Result



recovered image

Limitation

- Inherently white or grayish objects



(a)



(b)



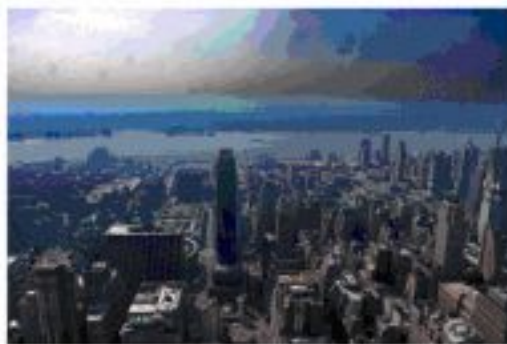
(c)

Limitation

- Haze imaging model is invalid



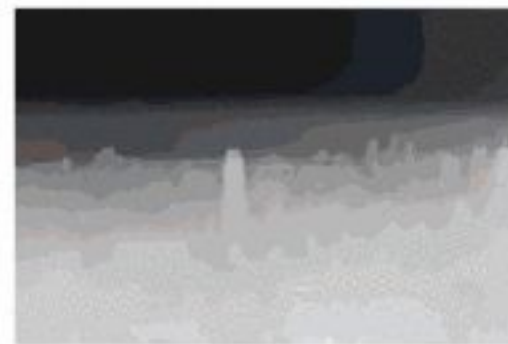
(a)



(b)



(c)



(d)

Summary

- Dark channel prior
 - A natural phenomenon
 - Very simple but effective
 - Put a bad image to good use

2

PART 02

基于C++的算法分析和验证

算法验证

在实际FPGA开发的过程中，单仿真需耗费很长的时间，往往需先用高级语言搭建一套完整的实现流程，方便FPGA开发时的验证。

而在图像处理操作中，可以应用C++去方便的调取OPEN_CV库中的一些接口函数，并且有一些便利的类可以使用，因此选用C++来对算法进行程序验证和分析

```
14 int main()
15 {
16     //imread 可以指定加载灰度图像或者是RGB图像
17     //imwrite 保存图像文件，类型由扩展名决定
18     Mat src = imread("../Single-Image-Haze-Removal-Using-Dark-Channel-Prior-main/pic/duck.jpg");
19
20     Mat dst;
21     Mat dark_channel_mat = dark_channel(src); //输出的是暗通道图像
22
23     int A = calculate_A(src, dark_channel_mat);
24     Mat tx = calculate_tx(src, A, dark_channel_mat);
25
26     //将原始图像转换为GRAY单通道灰度图
27     cvtColor(src, dst, COLOR_BGR2GRAY);
28     Mat tx_ = guidedfilter(dst, tx, 5, 0.001); //导向滤波后的tx, dst为引导图像
29     imshow("导向滤波", tx_);
30
31     Mat haze_removal_image = haze_removal_img(src, A, tx_);
32     imshow("透射率函数", tx);
33     imshow("原始图", src);
34     imshow("暗通道图", dark_channel_mat);
35     imshow("原始图", src);
36     imshow("去雾图", haze_removal_image);
37
38     waitKey(0);
39     return 0;
40 }
41
```

算法验证

获得暗通道图像

遍历原始图像的所有像素点，找到每个像素点三个通道中的最小值，将最小值组成单通道图。

```
35 //计算暗通道图像矩阵，针对三通道彩色图像
36 Mat dark_channel(Mat src)
37 {
38     //窗口的大小。这个对结果来说是个关键的参数，窗口越大，其包含暗通道的概率越大，暗通道也就越黑。我们不去
39     //从理论角度分析，从实践的效果来看，似乎窗口越大，去雾的效果越不明显
40     //滤波半径越大，最后去雾的图片中出现色斑几率越小，但是半径的大小影响效率
41     //暗通道图像均使用的窗口大小为5*5，即最小值滤波的半径为2像素。
42     int border = 2;
43     // 默认构造3个Mat (定义模板类)
44     std::vector<cv::Mat> rgbChannels(3);
45     //创建一个和原始图像src高和宽一致的8位无符号单通道的灰度图片容器,并且初始化图片为黑色0,创建一个空容器
46     Mat min_mat(src.size(), CV_8UC1, Scalar(0)), min_mat_expansion;
47     //创建一个和原始图像src高和宽一致的8位无符号单通道的灰度图片容器,并且初始化图片为黑色0
48     Mat dark_channel_mat(src.size(), CV_8UC1, Scalar(0));
49     //利用vector对象对原始图像进行通道分离 B G R
50     split(src, rgbChannels);
51     //遍历src的每个像素点，并比较每个像素点的 B G R 的像素值，找到最小值存放放到min_mat中去
52     for (int i = 0; i < src.rows; i++) //src.rows 二维数组的行数
53     {
54         for (int j = 0; j < src.cols; j++) //src.cols 二维数组的列数
55         {
56             int min_val = 0;
57             int val_1, val_2, val_3;
58             val_1 = rgbChannels[0].at<uchar>(i, j); //获取像素值B
59             val_2 = rgbChannels[1].at<uchar>(i, j); //获取像素值G
60             val_3 = rgbChannels[2].at<uchar>(i, j); //获取像素值R
61
62             min_val = std::min(val_1, val_2);
63             min_val = std::min(min_val, val_3); //获取B, G, R中最小的像素值
64             min_mat.at<uchar>(i, j) = min_val; //像素值赋值
65             //得到RGB分量中最小值组成的灰度图像
66         }
67     }
```



算法验证

获得暗通道图像

将获得的单通道图进行扩展，扩展范围等于滤波半径。对扩展前图像的每一个像素点进行滤波半径为 r 的均值滤波，最后得到暗通道图

```
98 //扩充灰度图像的边缘，得到min_mat_expansion。向四周扩展的范围刚好是滤波半径
99 //BORDER_REPLICATE表示扩充边缘的方式是边界复制扩充
100 //假如100*100的一张图，扩展后就成为104*104的一张图(扩展半径为2)
101 copyMakeBorder(min_mat, min_mat_expansion, border, border, border, border, BORDER_REPLICATE); //扩充图像边缘，min
102
103 //遍历原始图像的大小 (2-102) * (2-102) == 100*100
104 for (int m = border; m < min_mat_expansion.rows - border; m++) // min_mat_expansion.rows 二维数组的行数
105 {
106     for (int n = border; n < min_mat_expansion.cols - border; n++) // min_mat_expansion.cols 二维数组的列数
107     {
108         Mat imageROI;
109         int min_num = 256;
110         //掩膜是滤波半径加1，即：2*border+1。在这个大小(5*5)下的进行最小值滤波
111         //以这个像素点为中心，以border为半径，获取 (2 * border + 1) * (2 * border + 1) 的正方形像素区域
112         imageROI = min_mat_expansion(Rect(n - border, m - border, 2 * border + 1, 2 * border + 1));
113         //最小值滤波：用邻域范围(5*5)内最小值去替换邻域中心的值。
114         for (int i = 0; i < imageROI.rows; i++)
115         {
116             for (int j = 0; j < imageROI.cols; j++)
117             {
118                 int val_roi = imageROI.at<uchar>(i, j);
119                 min_num = std::min(min_num, val_roi); //找到这个区域里边最小的像素值 min_num
120             }
121         }
122         //将这个区域内中心像素点的值替换为找到的这个最小值
123         dark_channel_mat.at<uchar>(m - border, n - border) = min_num;
124         //整幅图遍历完毕，生成暗通道图像
125     }
126 }
127 return dark_channel_mat;
128 //return min_mat;
```



算法验证

计算全球大气光成分 A

遍历暗通道图每一个像素点的像素值, 存储的过程并完成从大到小排序, 同时包含每个像素点的位置信息。

```
// 全球大气光成分A
// 求出原始有雾图像的暗通道图像, 在暗通道图像中找到像素值最高的、占总像素数
// 目前0.1%的像素点位置, 这些位置中原始图像像素值最高的那个像素点, 就是大气光A
int calculate_A(Mat src, Mat dark_channel_mat)
{
    // 默认构造3个Mat容器
    std::vector<cv::Mat> rgbChannels(3);
    // 利用vector对象对原始图像进行通道分离 B G R
    split(src, rgbChannels);
    // 准备存储原图的图结构, STL实现的map本质是一棵红黑树, 它具备自动排序的能力。
    // 这点将省去排序的工作, Point同时存储了像素的位置。
    // 定义一个键的类型为int类型、实值类型为point的空map pair_data 容器。
    map<int, Point> pair_data;
    map<int, Point>::iterator iter;
    // 定义一个点类型的容器cord
    vector<Point> cord;
    int max_val = 0;
    // cout << dark_channel_mat.rows << " " << dark_channel_mat.cols << endl;
    // 遍历暗通道图像, 得到每一个像素点的位置信息和像素值信息, 并存入pair_data容器中,
    // 并且会对这些像素点的值从小到大进行排序, 最后一个元素也就是像素值最大的。
    for (int i = 0; i < dark_channel_mat.rows; i++)
    {
        for (int j = 0; j < dark_channel_mat.cols; j++)
        {
            int val = dark_channel_mat.at<uchar>(i, j); // 获取每一个点的像素值
            Point pt; // 定义一个坐标点
            pt.x = j;
            pt.y = i;
            pair_data.insert(make_pair(val, pt)); // 存入坐标点的位置信息
            // 当我们向一颗极其严格的平衡二叉
        }
    }
}
```

算法验证

计算全球大气光成分 A

找出存储的像素点值最大的像素点的位置信息，与之对应找到原图像该位置的像素点。比较三个通道的像素值，即可得到A的值。

```
161  
162 //遍历这棵二叉树，  
163 for (iter = pair_data.begin(); iter != pair_data.end(); iter++)  
164 {  
165     //取出Hash表中的value，而不是key。(指像素点的位置信息)  
166     //cout << iter->first << endl;  
167     cord.push_back(iter->second);  
168 }  
169 Point tmp = cord[cord.size() - 1];  
170 int val_1, val_2, val_3;  
171 val_1 = rgbChannels[0].at<uchar>(tmp.y, tmp.x); //获取 (x, y) 坐标位置像素点的B通道像素值  
172 val_2 = rgbChannels[1].at<uchar>(tmp.y, tmp.x); //获取 (x, y) 坐标位置像素点的G通道像素值  
173 val_3 = rgbChannels[2].at<uchar>(tmp.y, tmp.x); //获取 (x, y) 坐标位置像素点的R通道像素值  
174 max_val = std::max(val_1, val_2);  
175 max_val = std::max(max_val, val_3);  
176  
177 return max_val; //A的值求出来了。  
178 }  
179
```

算法验证

计算透射率t

输入暗通道图和A的值，带入对应的计算公式得到透射率图

```
181
182 //透射率t(x)
183 Mat calculate_tx(Mat& src, int A, Mat& dark_channel_mat)
184 { //定义两个矩阵
185   Mat dst; //是用来计算t(x)
186   Mat tx;
187   float dark_channel_num;
188   dark_channel_num = A / 255.0;
189   //将暗通道图像转化为32位单精度浮点单通道的图像
190   //由于 CV_8UC1 的值范围为0到255，而 CV_32FC1类型的取值仅为0.0到1.0。
191   //所以转换时不能直接用默认的尺度因子1，而应该用1/255.0。
192   dark_channel_mat.convertTo(dst, CV_32FC1, 1 / 255.0); //用来计算t(x)
193   // imshow("dst", dst);
194   // waitKey(0);
195   dst = dst / dark_channel_num;
196   tx = 1 - 0.95 * dst;
197   //修正因子影响去雾的效果，这个因子越大去雾越好，但是也会使得图像变得失真
198   //ω具有明显的意义，其值越小，去雾效果越不明显
199   return tx;
200 }
201
```



算法验证

导向滤波优化透射率图t

算法步骤如下：

1. 计算引导图G的积分图和平方积分图
2. 计算输入图像P的积分图，P和G的乘积积分图
3. 用上两步得出的积分图计算P和G的均值，G的方差，P和G的协方差，窗口半径为r
4. 然后用下式计算系数图A和B
5. 计算A和B的积分图
6. 计算A和B的窗口半径r的均值，并用下式计算输出图Q

$$b_k = \bar{p}_k - \mu_k a_k$$
$$q_i = \frac{1}{|W|} \sum_{k|i \in W_k} (a_k g_i + b_k)$$
$$q_i = \bar{a}_i g_i + \bar{b}_i$$

```
28 int nCols = srcImage.cols;
29 Mat boxResult;
30 //步骤一：计算均值
31 boxFilter(Mat::ones(nRows, nCols, srcImage.type()), boxResult, CV_32FC1, Size(r, r));
32 //imshow("boxResult", boxResult);
33 //waitKey(0);
34 //生成导向均值mean_I
35 Mat mean_I;
36 boxFilter(srcImage, mean_I, CV_32FC1, Size(r, r));
37 //生成原始均值mean_p
38 Mat mean_p;
39 boxFilter(srcClone, mean_p, CV_32FC1, Size(r, r));
40 //生成互相关均值mean_Ip
41 Mat mean_Ip;
42 boxFilter(srcImage.mul(srcClone), mean_Ip, CV_32FC1, Size(r, r)); //互相关表示两个向量对
43 //生成自相关均值mean_II
44 Mat mean_II;
45 //应用盒滤波器计算相关的值
46 boxFilter(srcImage.mul(srcImage), mean_II, CV_32FC1, Size(r, r)); //自相关矩阵定义为样
47 //步骤二：计算相关系数
48 Mat var_I = mean_II - mean_I.mul(mean_I); //方差
49 Mat cov_Ip = mean_Ip - mean_I.mul(mean_p); //协方差
50 //步骤三：计算参数系数a, b
51 Mat a = cov_Ip / (var_I + eps);
52 Mat b = mean_p - a.mul(mean_I);
53 //步骤四：计算系数a\b的均值
54 Mat mean_a;
55 boxFilter(a, mean_a, CV_32FC1, Size(r, r));
56 mean_a = mean_a / boxResult;
57 Mat mean_b;
58 boxFilter(b, mean_b, CV_32FC1, Size(r, r));
59 mean_b = mean_b / boxResult;
60 //步骤五：生成输出矩阵
61 Mat resultMat = mean_a.mul(srcImage) + mean_b;
62 return resultMat;
```


算法验证

导向滤波优化透射率图t



算法验证

去雾

对原始图像每一个像素点的每一个通道值都程序中的公式处理, 最后将三通道合并, 得到去雾后的图像。

```
205 //创建一个和原图像大小相等的RGB彩色图像 mat矩阵
206 Mat result_img(src.rows, src.cols, CV_8UC3);
207 //创建两个mat类型的变量组, 一组三个元素
208 vector<Mat> srcChannels(3), resChannels(3);
209 //利用vector对象对原始图像进行通道分离 B G R
210 split(src, srcChannels);
211 //利用vector对象对3通道彩色图像矩阵进行分离 获得3幅原图像的BGR三通道图
212 split(result_img, resChannels);
213
214 for (int i = 0; i < src.rows; i++) //src.rows 二维数组的行数
215 {
216     for (int j = 0; j < src.cols; j++) //src.cols 二维数组的列数
217     {
218         for (int m = 0; m < 3; m++)
219         {
220             //循环获取单个通道中每一个像素的像素值
221             int value_num = srcChannels[m].at<uchar>(i, j);
222             //取出相对应像素位置的透射率t(x)
223             float max_t = tx.at<float>(i, j);
224             //如果透射率太低就进行限幅
225             if (max_t < 0.1)
226             {
227                 max_t = 0.1;
228             }
229             //J=(I-A)/t+A value_num==I==原始图像R/G/B三个通道的像素值,
230             //J==resChannels[m]是处理完的结果图
231             resChannels[m].at<uchar>(i, j) = (value_num - A) / max_t + A;
232         }
233     }
234 }
235 //把处理好的3个单通道进行合并得到去雾后的图像
236 merge(resChannels, result_img);
237
238 return result_img;
239 }
```



3

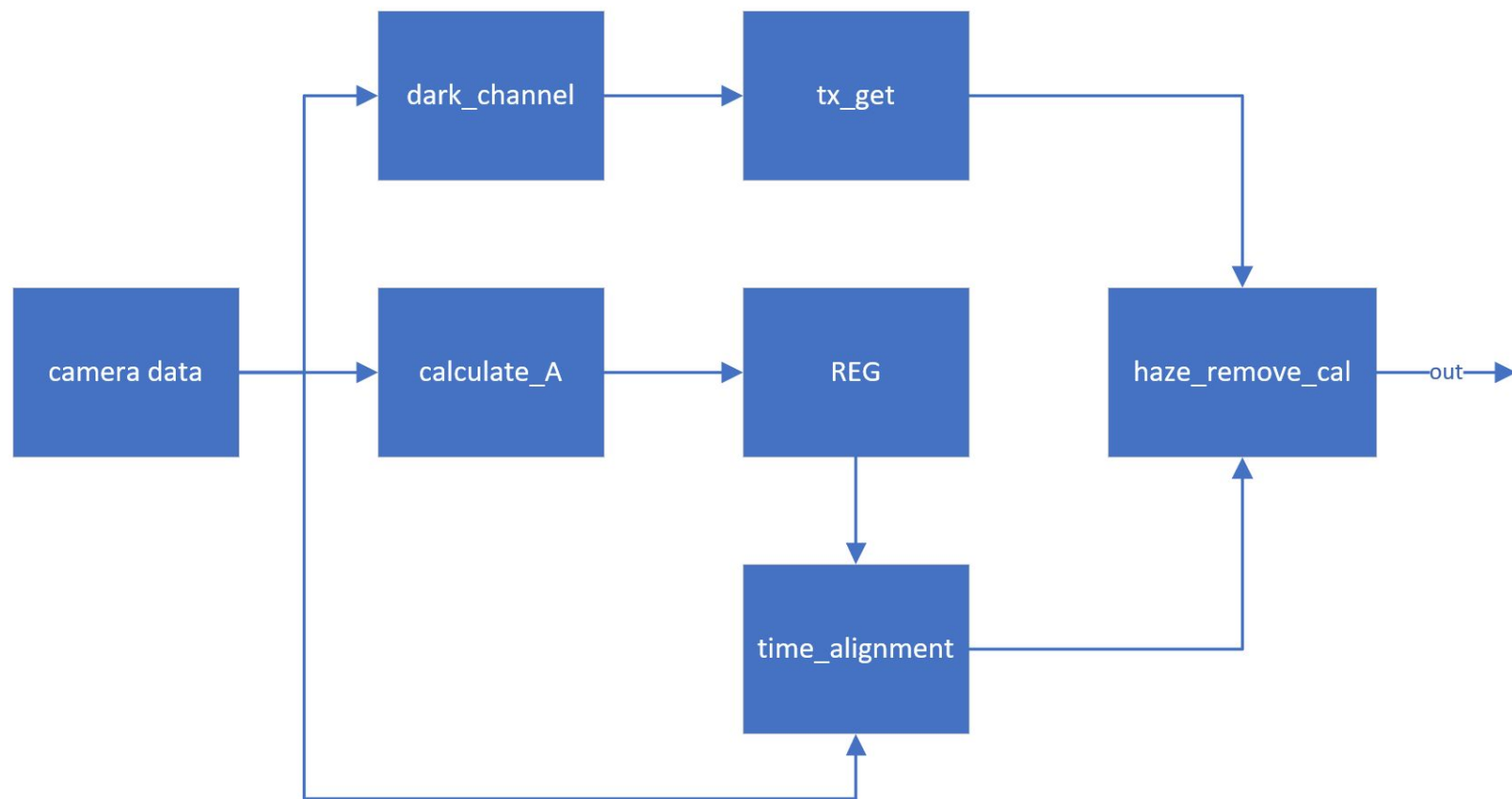
PART 03

硬件实现架构及仿真分析

实现架构及仿真分析

algorithm flow

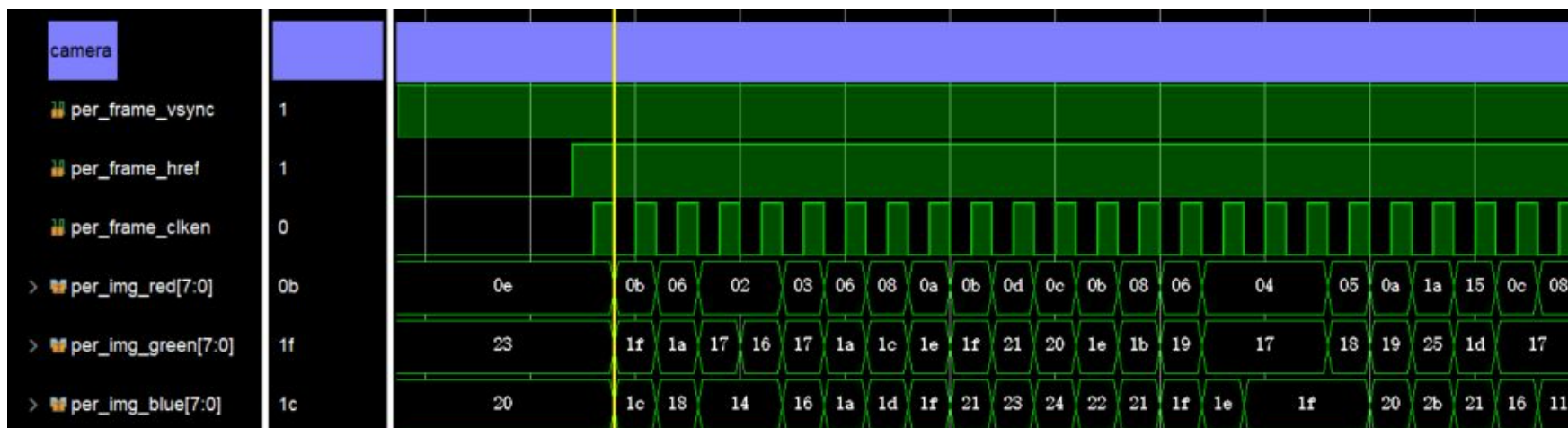
- camera_data_make
- dark_channel
- calculate_A
- tx_get
- time_alignment
- haze_remove_cal



实现架构及仿真分析

algorithm flow

- camera_data_make

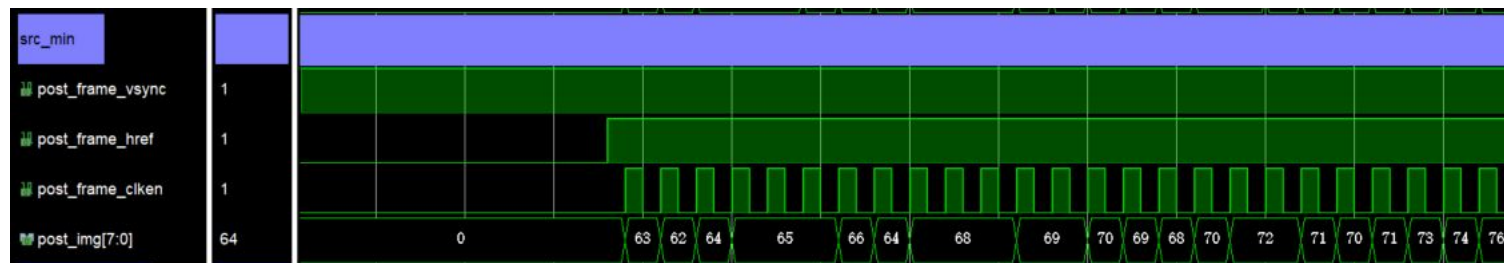


camera data stream

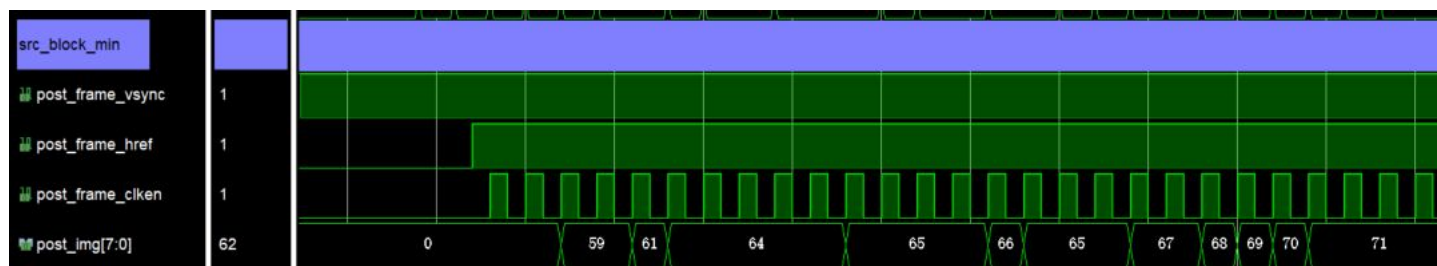
实现架构及仿真分析

algorithm flow

- dark_channel



find min value of RGB channel

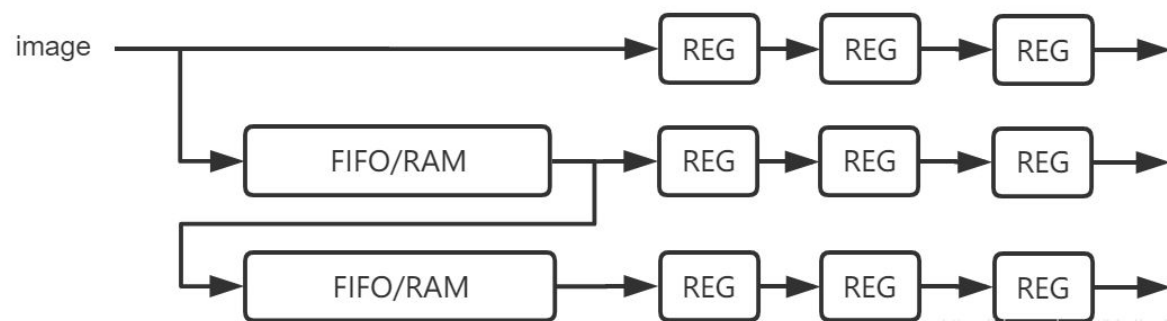
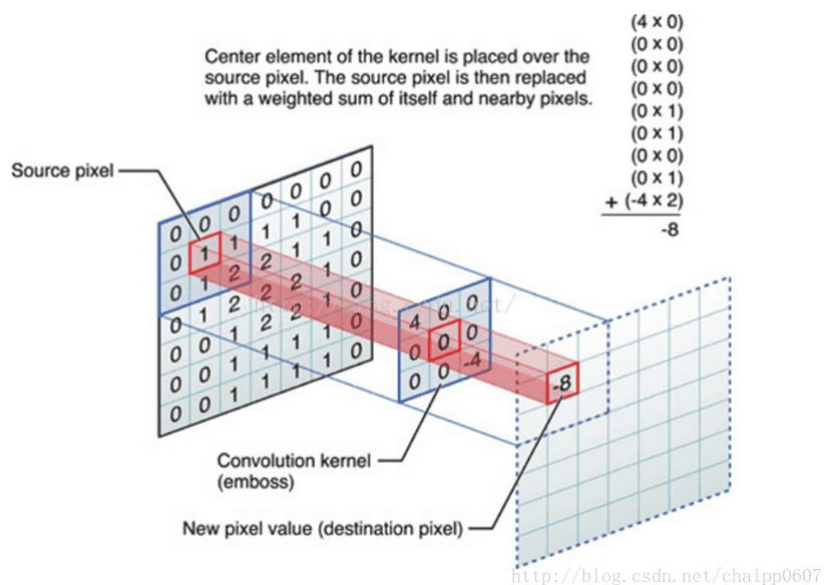


min value filter

实现架构及仿真分析

algorithm flow

- dark_channel



convolution operator

实现架构及仿真分析

algorithm flow

- calculate_A

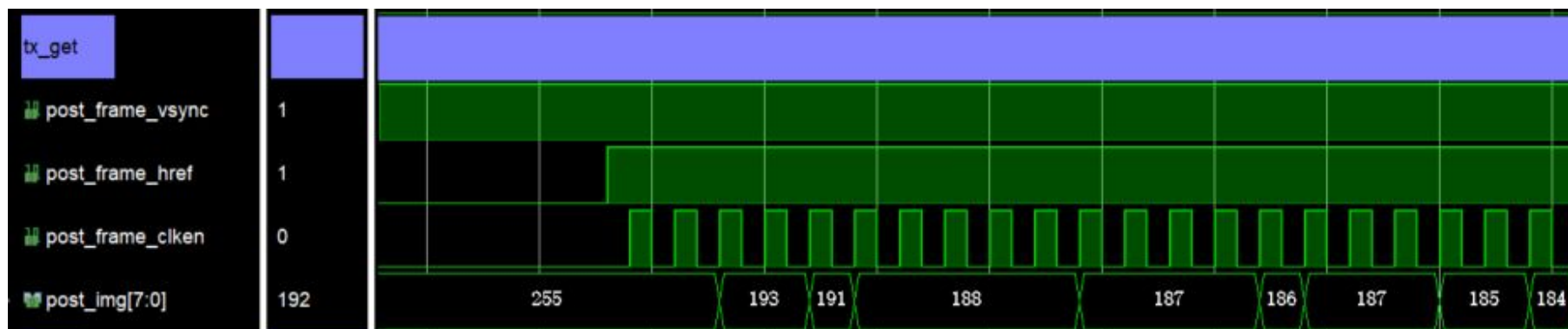


previous frame value

实现架构及仿真分析

algorithm flow

- tx_get

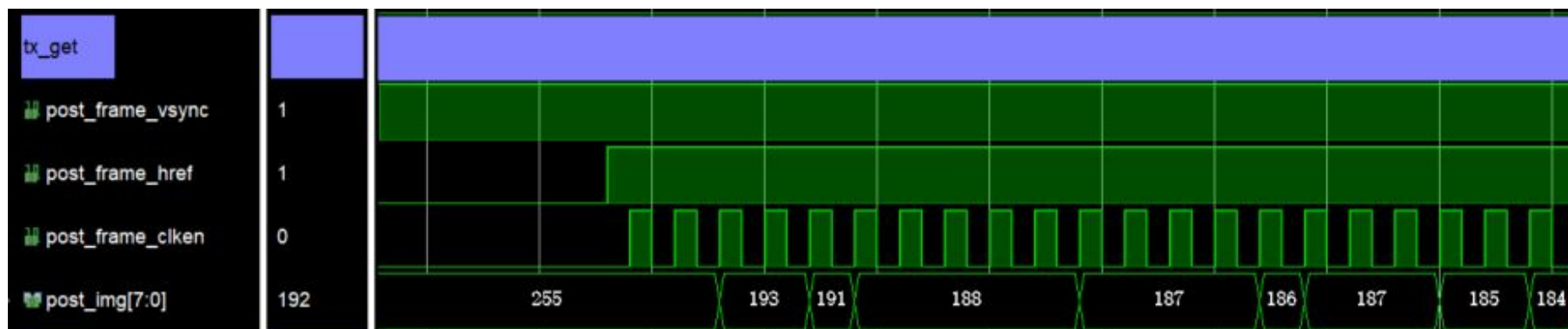


transmission value

实现架构及仿真分析

algorithm flow

- **time_alignment**



transmission value

实现架构及仿真分析

algorithm flow

- haze_remove_cal



final result

实现架构及仿真分析

Simulation Result



haze image



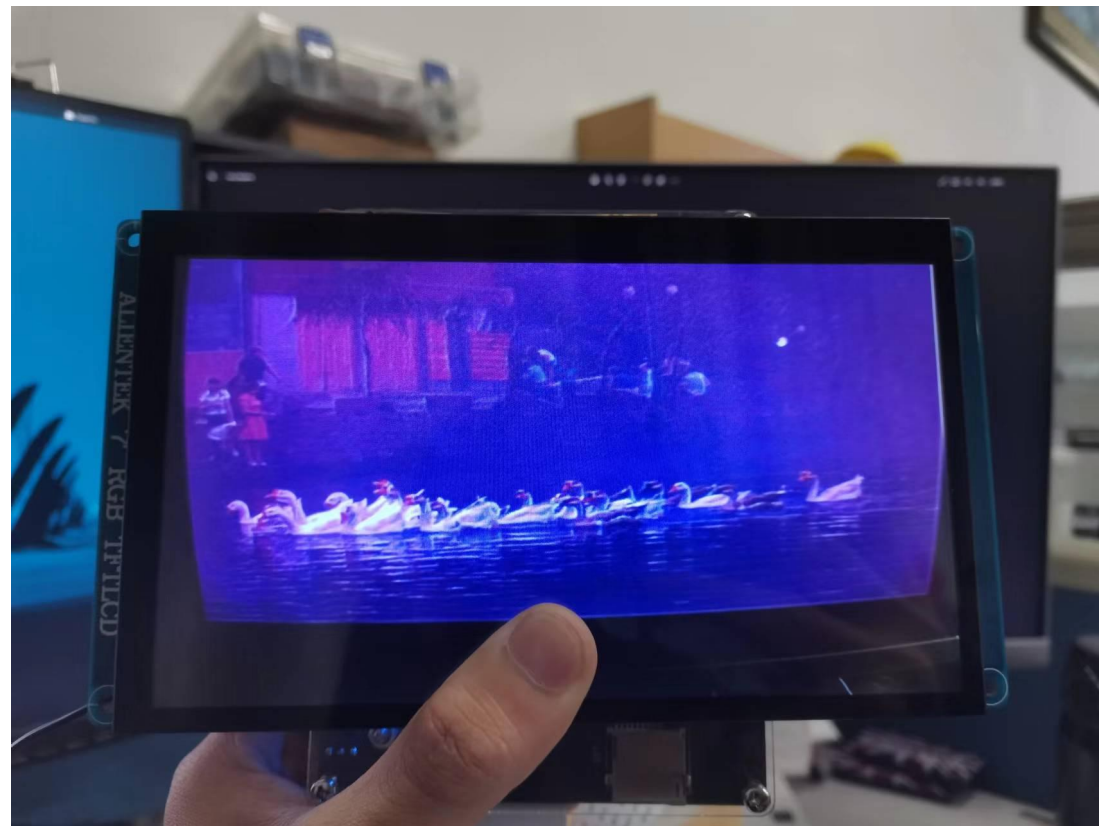
scene radiance

实现架构及仿真分析

Site test



haze image



scene radiance

4

PART 04

Reference

Reference

K. He, J. Sun and X. Tang, "Single Image Haze Removal Using Dark Channel Prior," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 12, pp. 2341-2353, Dec. 2011, doi: 10.1109/TPAMI.2010.168.

K. He, J. Sun and X. Tang, "Guided Image Filtering," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 35, no. 6, pp. 1397-1409, June 2013, doi: 10.1109/TPAMI.2012.213.

Thank You